34

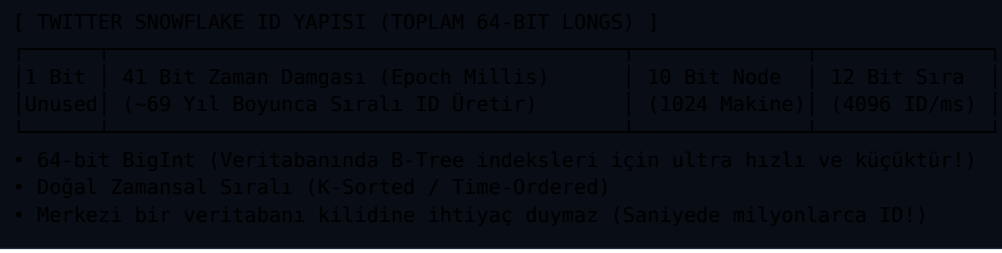
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

# Dağıtık Benzersiz ID Üretimi: Twitter Snowflake, UUIDv7 ve Ticket Server

Distributed Unique ID Generation: Twitter Snowflake & UUIDv7

## 📐 Sistem Mimarisi & Veri Akış Şeması

## 📐 Architecture & Data Flow Diagram




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir doğum hastanesini düşünün: Her doğan bebeğe benzersiz bir kimlik numarası verilmelidir. Eskiden tek bir memur sırayla 1, 2, 3… diye numara verirdi (Auto-Increment). Ama dünyada aynı anda 100 hastanede bebek doğuyorsa hepsi Ankara'daki o tek memura telefon açamaz (Darboğaz ve çöküş!). Twitter Snowflake ise her hastaneye özel bir formül verir: 'Önce şu anki saati yaz, yanına kendi hastane kodunu ekle, yanına da o saniyedeki bebek sırasını ekle!' Böylece hiçbir hastane birbirine telefon etmeden saniyede milyonlarca benzersiz ve kronolojik olarak sıralı kimlik numarası üretir!

### 🇬🇧 ENGLISH (ELI5)

Imagine assigning national birth certificate numbers. A naive approach is a single bureaucrat sitting in the capital stamping 1, 2, 3… sequentially (Auto-Increment ID). But if 1,000 maternity hospitals across the nation deliver babies simultaneously, that single bureaucrat causes nationwide traffic jams! Twitter Snowflake equips every hospital with a decentralized formula: 'Stamp the current millisecond timestamp, append your hospital ID, append a millisecond sequence counter!' No hospital coordinates with another, yet every generated ID is universally unique and strictly chronological!

## 🔍 MAANG Seviyesi Teknik Derinlik

### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık mimarilerde ID gereksinimleri: Benzersizlik, zamana göre sıralanabilirlik (K-Sortable / B-Tree indeks dostu), 64-bit tamsayı (RAM ve disk tasarrufu). Yöntemler: 1) UUIDv4: 128-bit rastgeledir; zamana göre sıralı değildir, B-Tree indekslerinde sayfa parçalanması (Page Fragmentation) yaratır. 2) UUIDv7: 128-bit'in ilk 48 bitini zaman damgası yaparak sıralı hale getirmiştir. 3) Twitter Snowflake: 64-bit'lik endüstri standardıdır: 1 bit işaret (0), 41 bit zaman damgası (69 yıllık ömür), 10 bit makine/veri merkezi ID'si (1024 bağımsız düğüm), 12 bit sıra numarası (Aynı milisaniyede düğüm başına 4096 ID). Tek bir düğüm saniyede 4 milyon üzerinde sıralı ID üretebilir; sıfır ağ çağrısı, sıfır kilit!

## 🔍 MAANG-Level Technical Deep-Dive

### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed ID generation requirements: 64-bit integer density, time-ordering (K-sortable to preserve B-Tree append locality), and zero network coordination. 1) UUIDv4: 128-bit completely random; destroys B-Tree index cache efficiency via catastrophic page splits. 2) UUIDv7: Modern 128-bit standard embedding a 48-bit millisecond timestamp prefix for sequential B-Tree friendliness. 3) Twitter Snowflake: The 64-bit gold standard: 1 unused sign bit, 41 bits millisecond epoch timestamp (69 years span), 10 bits machine/datacenter identifier (1024 unique generator nodes), 12 bits sequence counter (4096 IDs per ms per node). A single node generates 4.096 million unique, sortable, collision-free IDs per second in pure local CPU memory without network RPCs.

### ⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Gerçek Twitter Snowflake 64-Bit ID Üreteci Implementasyonu
public class SnowflakeIdGenerator {
    private static final long EPOCH = 1704067200000L; // Özel Başlangıç (1 Ocak 2024)
    private static final long WORKER_ID_BITS = 10L;
    private static final long SEQUENCE_BITS = 12L;

    private static final long MAX_WORKER_ID = -1L ^ (-1L << WORKER_ID_BITS); // 1023
    private static final long MAX_SEQUENCE = -1L ^ (-1L << SEQUENCE_BITS);   // 4095

    private final long workerId;
    private long sequence = 0L;
    private long lastTimestamp = -1L;

    public SnowflakeIdGenerator(long workerId) {
        if (workerId > MAX_WORKER_ID || workerId < 0) {
            throw new IllegalArgumentException("Worker ID 0 ile " + MAX_WORKER_ID + " arasında
olmalıdır");
        }
        this.workerId = workerId;
    }

    public synchronized long nextId() {
        long currentTimestamp = System.currentTimeMillis();

        if (currentTimestamp == lastTimestamp) {
            sequence = (sequence + 1) & MAX_SEQUENCE;
            if (sequence == 0) {
                // Aynı milisaniyede 4096 sınırına ulaşıldı -> Sonraki milisaniyeyi bekle!
                while (currentTimestamp <= lastTimestamp) {
                    currentTimestamp = System.currentTimeMillis();
                }
            }
        } else {
            sequence = 0L;
        }

        lastTimestamp = currentTimestamp;

        return ((currentTimestamp - EPOCH) << (WORKER_ID_BITS + SEQUENCE_BITS))
                | (workerId << SEQUENCE_BITS)
                | sequence;
    }
}

void main() {
    var generator = new SnowflakeIdGenerator(7); // Düğüm ID: 7
    for (int i = 0; i < 3; i++) {
        IO.println("Snowflake 64-bit ID: " + generator.nextId());
    }
}
```

### ⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

**🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ**

Snowflake algoritmasının tek zayıf noktası 'Saat Geriye Kayması' (Clock Moving Backwards) durumudur! Eğer NTP sunucu saatini 5 milisaniye geriye çekerse, daha önce üretilmiş bir ID tekrar üretilebilir (ID Çarpışması!). Gerçek implementasyonlarda saat geri giderse hata fırlatılmalı veya saat yakalanana kadar beklenmelidir.

### ⚠️ Interview Trap & Bottleneck Alert

**🇬🇧 INTERVIEW TRAP & BOTTLENECK**

The critical Achilles' heel of Snowflake is NTP clock skew moving backwards. If system time rolls backward by 5ms, duplicate IDs can collide! Production implementations must throw exceptions or sleep until current time catches up with `lastTimestamp`.

### 🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Snowflake ID'leri 64-bit boyutuyla veritabanı indekslerinde minimum yer kaplar.
- Doğal zaman sıralaması (K-Sorted) sayesinde en son kayıtlar daima tablonun sonuna eklenir.
- Merkezi bir veritabanı kilit servisine ihtiyaç duymadan yerel olarak üretilir.

### 🎯 Key Takeaways for the Interview

- 64-bit Snowflake integers maximize B-Tree clustering index density and speed.
- Time-ordered IDs prevent random B-Tree page splits and fragmentation.
- Snowflake nodes operate autonomously without distributed locks or network latency.